



Open-Source Multimedia Retrieval with vitrivr-engine

Ralph Gasser

University of Basel
Basel, Switzerland
ralph.gasser@unibas.ch

Rahel Arnold

University of Basel
Basel, Switzerland
rahel.arnold@unibas.ch

Laura Rettig

University of Basel
Basel, Switzerland
laura.rettig@unibas.ch

Heiko Schuldt

University of Basel
Basel, Switzerland
heiko.schuldt@unibas.ch

Raphael Waltenspül

University of Basel
Basel, Switzerland
raphael.waltenspuel@unibas.ch

Luca Rossetto

Dublin City University
Dublin, Ireland
luca.rossetto@dcu.ie

Abstract

The vitrivr multimedia retrieval stack has been shown to be versatile and effective in many areas and application domains. In this paper, we present vitrivr's new retrieval engine, the vitrivr-engine, a flexible and highly configurable information extraction and query processing solution with support for a broad range of media types.

CCS Concepts

• **Information systems** → **Multimedia and multimodal retrieval**; *Image search*; *Video search*; *Speech / audio search*.

Keywords

Multimedia Retrieval, Retrieval Engine, Open Source Software

ACM Reference Format:

Ralph Gasser, Rahel Arnold, Laura Rettig, Heiko Schuldt, Raphael Waltenspül, and Luca Rossetto. 2025. Open-Source Multimedia Retrieval with vitrivr-engine. In *Proceedings of the 33rd ACM International Conference on Multimedia (MM '25)*, October 27–31, 2025, Dublin, Ireland. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3746027.3756874>

1 Introduction

The ever increasing volume of multimedia data requires effective retrieval mechanisms to be useful. To help deal with this need for effective retrieval solutions, we introduced *vitrivr* [5], our retrieval stack with support for a broad range of media types, query-, and interaction patterns as open-source software several years ago. The vitrivr stack consists, in its common configuration, of three components: a database storing retrieval-relevant feature information, a feature extraction and query processing engine, and different user interfaces for query formulation and results exploration in various modalities (Web, VR, AR). While many of these components have seen replacements over the years, the feature extraction and query processing engine, *Cineast* [4], has remained a central and rather static component of the vitrivr stack throughout its development. Although *Cineast* has served us well for many years, its

fixed-function pipeline architecture increasingly became a limitation to the exploration of novel retrieval approaches, especially with regard to recent advances in machine learning and AI.

In this paper, we present the *vitrivr-engine*, first introduced in [1] and under active development since then, as a successor to *Cineast*. Like its predecessor, the vitrivr-engine is designed as a platform-independent application and is released as open-source software. Unlike its predecessor, the vitrivr-engine offers flexible and highly configurable feature extraction and query processing pipelines that can be easily adapted to a broad range of use cases and information needs. Currently, we have used vitrivr-engine successfully in several evaluation campaigns [1, 3, 6, 7] and for a range of different scientific and educational purposes such as the *XReco* projects. The project repository, including all the source code and technical documentation, can be found on GitHub¹ and is freely available under a permissive open source license. All aspects described in this paper relate to the main branch of the project and the latest release version (version 0.1.10).

The remainder of this paper is structured as follows: Section 2 provides a brief overview of the high-level design of vitrivr-engine. Section 3 outlines its capabilities. Section 4 lists the necessary steps to use vitrivr-engine, including references to more detailed documentation. Finally, Section 5 concludes with an outlook.

2 System Design

vitrivr-engine is written in Kotlin and runs in a Java Virtual Machine (JVM). It uses a modular architecture and a flexible data layout, ensuring applicability to a wide range of use cases.

2.1 Code Architecture

The entire application follows a modular architecture with several modules for different purposes. The *core module* defines the fundamental data types and the interfaces of relevant structures such as operators and operator pipelines. The index and query modules define the base structures for information extraction and retrieval. We use the JVM's *ServiceLoader* facility to provide implementations of all the different interfaces through additional modules. This internal abstraction enables the seamless exchange or addition of almost any aspect of the vitrivr-engine, including but not limited to adapters for storage backends, decoders for new media types or feature extraction operations. Some defaults are bundled with the main code base, but can also be provided by third-party developers.

¹See <https://github.com/vitrivr/vitrivr-engine>



This work is licensed under a Creative Commons Attribution 4.0 International License. *MM '25*, Dublin, Ireland

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2035-2/2025/10
<https://doi.org/10.1145/3746027.3756874>

The default modules include different such implementations, including connections for different feature stores, such as Cottontail DB [2] or PostgreSQL with the pgvector² extension, decoders, and features for typical media types (e.g., videos or images) and application scenarios. Feature transformations can be implemented directly in native Kotlin or accessed through external services and APIs (e.g., TorchServe³).

A *server module* implements all relevant logic to run vitivr-engine as a standalone application. Provides RESTful APIs and a command-line interface (CLI) to access all relevant functionality. However, its modular architecture also makes it possible to use vitivr-engine (or parts thereof) as a library or to implement custom APIs, should a use case require it.

2.2 Data Architecture

vitrivr-engine uses the notion of a *schema* to organize all data relevant to the retrieval process. A schema has a name and an arbitrary number of named *fields*, each representing an attribute of a document in the collection. A field is commonly owned by an associated *analyzer*, i.e., a feature transformation function or other processing operator. Each field is strictly typed and can contain scalar, vector, and composite types.

Schemas can be easily defined using JSON. We refer to the Wiki for examples.⁴ There is no explicit limit to the number of concurrent schemas that can be made available to vitivr-engine. The common way to use schemas is to define one schema per collection. This way, a single vitivr-engine instance can support retrieval in multiple collections, even if the collections are structured differently or use different features.

Data collections in vitivr-engine are represented as a collection of *retrievables*, which form the base unit of retrieval. A retrievable can represent anything, but typically it's a proxy for a document (e.g., an image or video file) or a part thereof (e.g., a temporal segment of a video file). Each retrievable has a unique identifier and can have relations to other retrievable to model these dependencies. Most importantly, however, every retrievable holds and is described by a selection of *descriptors*, which represent the retrievable and its content in some aspect. Typically, descriptors are backed by fields and they hold the data type described in it.

In Figure 1 we provide an example of how this data model can be employed. The example was used during the Lifelog Search Challenge (LSC) 2024 [7], to model lifelog data.

3 System Capabilities

vitrivr-engine is mainly used for two processes; information extraction and indexing of documents during an offline-phase, and query evaluation and document retrieval during an online-phase. In both phases, user-defined *pipelines* are used.

3.1 Information Extraction

vitrivr-engine offers a broad range of mechanisms to decode and ingest various media types. Once decoded, a document's content are represented as *retrievables*. Depending on the media type and

use case, a document can be represented by one retrievable or segmented into arbitrarily many *sub-retrievables*, such as in the case of video shots being related to the full video document. During decoding, the decoded content will be attached to the corresponding retrievables to make it available for downstream processing.

The processing pipeline is defined as an acyclic directed graph (DAG) where each node in the graph represents an operator able to perform work on a retrievable. The most common type of operator is the *analyzer*, which takes the information attached to a retrievable and generates one or multiple *descriptors*, which will also be attached. If the analyzer is backed by a *field* of the schema, the descriptor is persisted for later retrieval.

Some basic examples of such information extraction and document ingestion are bundled with the project. More examples can be found on our Wiki.⁵

3.2 Multimedia Retrieval

Queries in vitivr-engine are also expressed in the form of a directed acyclic graph that defines the way in which a query is to be executed. In addition to the execution graph, the query contains an arbitrary number of named inputs that contain typed information to be used during the query execution process. These *inputs* can contain text, scalar values, images, or other types, depending on the features to be used in the query. Nodes in the query graph can use these inputs as parameters for various purposes.

The most common use is as input for a *retriever* node, which is backed by a field. It can use the input to compare it against the information previously extracted and persisted in the field to produce a stream of retrievables. These retrievables can then be further processed by other nodes, to apply further filtering, lookup or to attach additional information, or select further retrievables by resolving a relationship. An example for the latter could be to obtain the video document a retrieved video segment belongs to. Multiple retrievable streams can also be combined using set operations or even more complex operators, such as filters for temporal dependencies for retrievables belonging to the same document.

Again, some basic examples of such queries are bundled with the project. More examples can be found on our Wiki.⁶

4 Getting Started

This section provides a brief overview on how to get started with vitivr-engine. For a more detailed guide, we refer to our Wiki.⁷

Building vitivr-engine

Running vitivr-engine requires a Java Runtime Environment (JRE) 21 or higher. It also requires a storage backend, such as Cottontail DB or PostgreSQL with the pgvector extension.

After cloning the repository from GitHub, build the application using the Gradle build system by running `./gradlew distZip`, which will produce a distribution bundle. Unzip the distribution bundle found in the distributions directory. Alternatively, the provided Docker image can be used.

²See <https://github.com/pgvector/pgvector>

³See <https://docs.pytorch.org/serve/>

⁴See <https://github.com/vitrivr/vitrivr-engine/wiki/Example#the-schema>

⁵See <https://github.com/vitrivr/vitrivr-engine/wiki/Example#the-ingestion>

⁶See <https://github.com/vitrivr/vitrivr-engine/wiki/Example#retrieval>

⁷See <https://github.com/vitrivr/vitrivr-engine/wiki/Getting-Started>

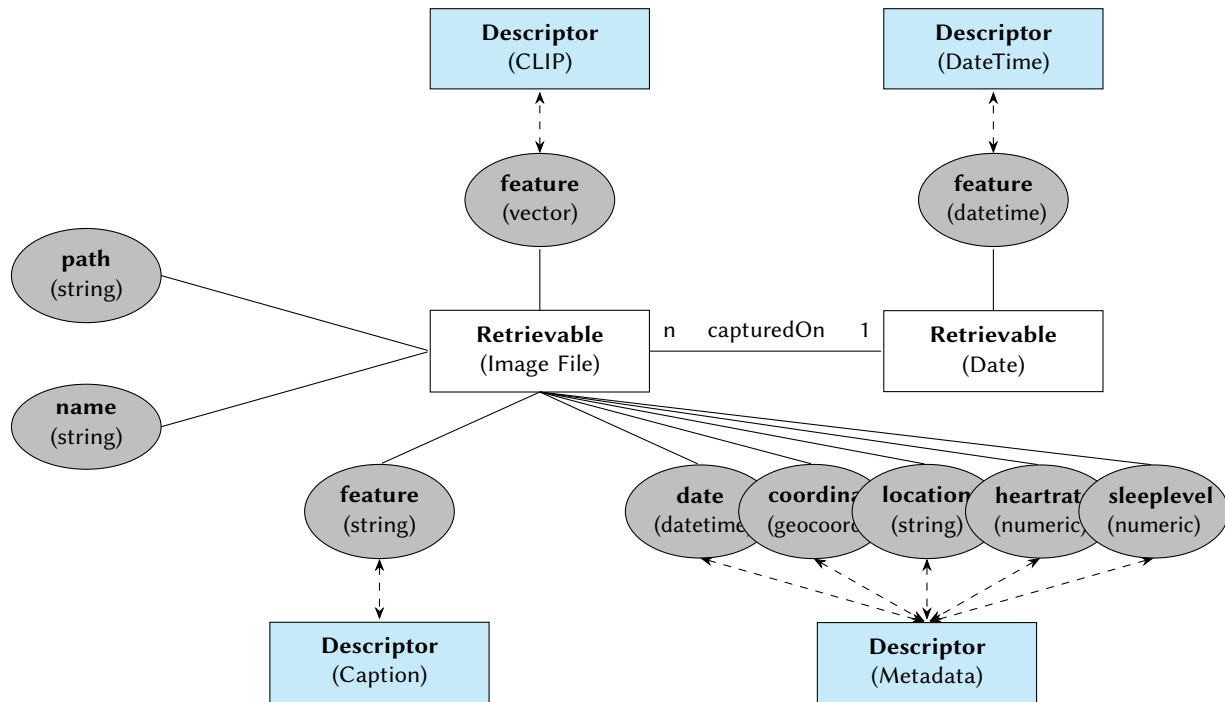


Figure 1: Simplified representation of the lifelog data in vitivr-engine [7]. We use two different retrievable to describe an individual image in the lifelog and the point in time during which the lifelog entry was created. Both retrievables are described by different descriptors.

Creating a Schema

Create a file called `schema.json` to define a schema or multiple schemas. A schema definition consists of a unique name, the database connection, the resolvers, and all the fields. Resolvers are used to locate files across persistent storage. In this example, we use the name `my-schema`. For more details on how to configure a schema, please refer to the Wiki.⁸

Ingestion

To ingest a collection of media items, an ingestion pipeline needs to be defined. Basically, the pipeline definition is a serialized version of the DAG in JSON. For detailed information on how to define a pipeline, we refer to the Wiki.⁹ After defining a pipeline, run the binary found in the `bin` directory of the unpacked application bundle. This will start vitivr-engine in standalone mode and bring up the CLI. Before any content can be ingested, the previously defined schema must be initialized. This can be done by typing `my-schema init` where `my-schema` is the name of the previously defined schema. The extraction pipeline (in this example assumed to be stored in a file with the name `pipeline.json`) can then be started by typing `my-schema extract -c /path/to/pipeline.json`. It is also possible to link extraction pipelines directly in the schema and refer to them by name instead.

⁸See <https://github.com/vitrivr/vitrivr-engine/wiki/Documentation#schema>

⁹See <https://github.com/vitrivr/vitrivr-engine/wiki/Documentation#ingestion>

Retrieval

For query evaluation, vitivr-engine offers a RESTful API compliant with the OpenAPI standard. The endpoint for querying can be accessed via `/api/my-schema/query` and expects the query definition as a JSON object by POST. The endpoint will return a list of retrievables with their associated information. Examples of different types of queries can be found on the Wiki.¹⁰

5 Outlook and Future Work

vitrivr-engine is still under active development and several extensions to its current capabilities are either planned or ongoing. Among those are extensions to the available features and support for additional neural inference frameworks for the easy integration of learned features (for example Ray). Other activities include support for cloud-native storage APIs and facilities for horizontal scaling to facilitate the processing of much larger collections. Additionally, the APIs for the specification of query and ingest pipelines are being streamlined to improve readability and reduce verbosity.

Acknowledgments

This work was partially funded by the Swiss National Science Foundation through Project [MediaGraph](#) (Grant Number 202125) and the Horizon Europe project [XR mEdia eCOsystem \(XReco\)](#), based on a grant from the Swiss State Secretariat for Education, Research and Innovation (contract no. 22.00268).

¹⁰See <https://github.com/vitrivr/vitrivr-engine/wiki/Example#retrieval>

References

- [1] Ralph Gasser, Rahel Arnold, Fynn Faber, Heiko Schuldt, Raphael Waltenspül, and Luca Rossetto. 2024. A New Retrieval Engine for Vitriivr. In *MultiMedia Modeling - 30th International Conference, MMM 2024, Amsterdam, The Netherlands, January 29 - February 2, 2024, Proceedings, Part IV (Lecture Notes in Computer Science, Vol. 14557)*. Springer, 324–331. doi:10.1007/978-3-031-53302-0_28
- [2] Ralph Gasser, Luca Rossetto, Silvan Heller, and Heiko Schuldt. 2020. Cottontail DB: An Open Source Database System for Multimedia Retrieval and Analysis. In *MM '20: The 28th ACM International Conference on Multimedia, Virtual Event / Seattle, WA, USA, October 12–16, 2020*. ACM, 4465–4468. doi:10.1145/3394171.3414538
- [3] Luca Rossetto and Ralph Gasser. 2025. Feature-Driven Video Segmentation and Advanced Querying with vitriivr-engine. In *MultiMedia Modeling - 31st International Conference on Multimedia Modeling, MMM 2025, Nara, Japan, January 8–10, 2025, Proceedings, Part V (Lecture Notes in Computer Science, Vol. 15524)*. Springer, 272–277. doi:10.1007/978-981-96-2074-6_32
- [4] Luca Rossetto, Ivan Giangreco, and Heiko Schuldt. 2014. Cineast: A Multi-feature Sketch-Based Video Retrieval Engine. In *2014 IEEE International Symposium on Multimedia, ISM 2014, Taichung, Taiwan, December 10–12, 2014*. IEEE Computer Society, 18–23. doi:10.1109/ISM.2014.38
- [5] Luca Rossetto, Ivan Giangreco, Claudiu Tanase, and Heiko Schuldt. 2016. vitriivr: A Flexible Retrieval Stack Supporting Multiple Query Modes for Searching in Multimedia Collections. In *Proceedings of the 2016 ACM Conference on Multimedia Conference, MM 2016, Amsterdam, The Netherlands, October 15–19, 2016*. ACM, 1183–1186. doi:10.1145/2964284.2973797
- [6] Loris Sauter, Ralph Gasser, Laura Rettig, Heiko Schuldt, and Luca Rossetto. 2024. General Purpose Multimedia Retrieval with vitriivr at LSC'24. In *Proceedings of the 7th Annual ACM Workshop on the Lifelog Search Challenge, LSC 2024, Phuket, Thailand, 10 June 2024*. ACM, 47–52. doi:10.1145/3643489.3661120
- [7] Loris Sauter, Ralph Gasser, Laura Rettig, Heiko Schuldt, and Luca Rossetto. 2024. General Purpose Multimedia Retrieval with vitriivr at LSC'24. In *Proceedings of the 7th Annual ACM Workshop on the Lifelog Search Challenge (Phuket, Thailand) (LSC '24)*. Association for Computing Machinery, New York, NY, USA, 47–52. doi:10.1145/3643489.3661120